CheckBridge — research proposal

Prepared 6 November 2026 · Juan Pablo Pérez Vélez

Question and relevance

When does a numerical checker improve a language model's answer, and when does a correct tool result create false confidence in assumptions that the tool never checked? A calculation can be valid while its interpretation is wrong. CheckBridge proposes a small, inspectable test of that distinction. The intended contribution is measurement and reproducibility for bounded tool-assisted oversight. The connection to foundational alignment is indirect: this does not solve recursive self-improvement, establish general model safety or demonstrate catastrophic-risk reduction. Research novelty remains unverified.

What exists now

The public, AI-assisted MIT prototype has three fixed cases, an experimental MCP stdio adapter and eleven passing local tests. It exposes numerical outputs, reference derivations, assumptions and individual checks. It does not execute supplied code or provide arbitrary file/network access. No live-model integration, real-model evaluation, independent domain review or security audit has been completed.
Source: https://github.com/juapperez/checkbridge

Proposed first twelve weeks

Expand to four cases and 24 original prompts with controlled wording, unit and missing-input variations. Compare Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct in direct-answer and tool-assisted conditions, with five repetitions: 480 main-comparison outputs. Pin model/runtime revisions; fix reference answers before the main comparison and retain a held-out subset. The checker must not be its own sole ground truth. A small exploratory integration test would precede the main run and be reported separately.
Weeks 1–4: references, harness and feasibility. Weeks 5–8: comparison and diagnostics. Weeks 9–12: scoring audit and reproducible release. These are proposed delivery windows to agree after funding, not a claim of full-time availability. I remain a university undergraduate and would work remotely from Colombia.

Measurement and failure cases

Record answer correctness, abstention, malformed calls, checker false positives/negatives, and explicit claims that a tool verified more than it did. Publish the scoring rubric and permitted model logs, including null results. Seek an independent scoring reviewer; none is recruited. Report descriptive results per task and condition without treating repeated outputs as independent evidence of broad reasoning ability. Risks include integration delays, simplistic fixtures, subjective confidence coding and a checker that adds little information.

What would change the research plan

The useful question is whether partial verification creates a repeatable, measurable oversight failure, not whether a few prompts can make a model fail. If the effect is highly wording-dependent, disappears with basic instructions, cannot be scored consistently, or is already adequately measured elsewhere, I would publish those limitations and revise or stop. If the pilot gives a reproducible signal, a possible months 4–6 extension would test whether explicitly marking the tool's unchecked assumptions changes unwarranted verification claims. That extension requires a separate scope, budget and funding decision. Over months 7–12, wider tasks or collaborators would be considered only if the earlier evidence supports them; no longer-term organization or commitment is claimed.

Support and budget

Request USD 6,000 for the initial pilot: implementation 3,600; original fixtures/reference calculations 800; independent review 400; bilingual documentation 400; compute 200; contingency 600. A reduced USD 3,000 version uses two cases, 12 prompts, one model, two conditions and five repetitions (120 main outputs) over eight weeks, allocated respectively 1,700/500/200/200/100/300. Author compensation is gross including applicable taxes, and these are deliverable estimates, not audited expenses or financial hardship. Compute is unbenchmarked. Research feedback and help finding an appropriate reviewer would also be useful; none is assumed.

Applicant, outputs and funding overlap

Juan Pablo Pérez Vélez is a software developer and Chemical Engineering undergraduate at Universidad Nacional de Colombia, Medellín, applying independently. His MIT expo-alphatab wrapper has seven passing tests and an existing iOS integration in Fretbox. Proposed outputs are original fixtures, a reproducible harness, permitted logs and an English/Spanish methods note. New code MIT, original documentation CC BY 4.0; preserve third-party licenses. Existing commercial apps are outside scope. Related cash and compute requests are pending, with no recorded awards/payments for this pilot; all overlapping costs would be disclosed and reconciled before accepting support. The prototype, tests and this proposal used AI assistance; this proposal was drafted by an AI assistant from verified records. No human-only authorship or university endorsement is claimed.